

Program 1: Understanding Constructor Chaining and Method Overloading

★ Instructions:

Analyse the code (in Program 1) below carefully. Then answer the questions that follow.

```
class Learn { refer to
    Learn() { object
        this("Smart"); // constructor chaining
        System.out.println("Default Constructor");
    }

    Smart
    Learn(String x) {
        System.out.println("Learn, " + x);
    }

    void show(String x) {
        System.out.println("Show: " + x);
    }

    void show(String x, String y) {
        System.out.println("Show: " + x + ", " + y);
    }

    public static void main(String[] args) {
        Learn obj = new Learn();
        obj.show("Focus");
    }
}
```

Program 1 : Learn.java

? Questions:

1. Which constructor is called first when `new Learn()` is created?
2. What is **constructor chaining** in this code? Which line shows this? *- this ("Smart")*
3. Identify which part of the code shows **method overloading**.
4. How many times is the **constructor invoked** when you run the program? *2 times*
5. What is the **output** of this program?

*3. void show (String x)
void show (String x, String y)*

*5. Learn, Smart
Default Constructor
Show: Focus*

Program 2 : Analysing Constructor and Method Overloading

Instructions:

Read the code below carefully. **Do not run the code.** Analyse the structure and answer the questions that follow.

```
class Calculator {
    int a, b;
    Calculator(int x, int y) {
        a = x;
        b = y;
        System.out.println("[Constructor] Sum: " + (a + b));
    }

    Calculator() {
        this(20, 10);
        System.out.println("[Constructor] Difference: " + (a - b));
    }

    int calculate(int x, int y) {
        return x + y;
    }

    int calculate(int x, int y, int z) {
        return x + y + z;
    }

    public static void main(String[] args) {
        Calculator calc = new Calculator();

        int result1 = calc.calculate(5, 15);
        int result2 = calc.calculate(1, 2, 3);

        System.out.println("[Method] Sum of 5 and 15: " + result1);
        System.out.println("[Method] Sum of 1, 2, and 3: " + result2);
    }
}
```

Program 2 : Calculator.java

? Questions:

1. How many **constructors** are defined in the class? *2*
2. Which constructor is called first when `new Calculator()` is used? *calculator(int x, int y)*
3. What is the purpose of `this(20, 10)` in the default constructor?
4. How is **constructor chaining** applied in this program?
5. Identify the **method overloading** in this code.
6. How many times are methods named **calculate** executed in `main()`? *2 times*
7. What is the expected output of the program?

3. It is used to call another constructor in the same class and perform constructor chaining

Concepts Covered:

4. Using this(20,10) in the default constructor

5. int calculate (int x, int y)

int calculate (int x, int y, int z)

7. Output :

[Constructor] Sum : 30

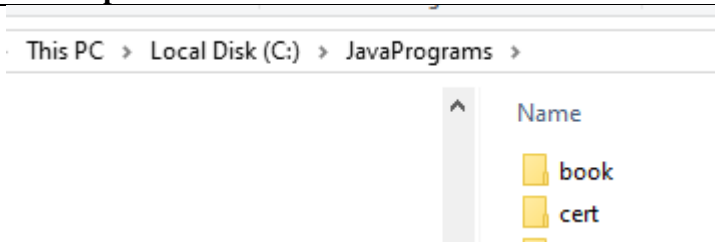
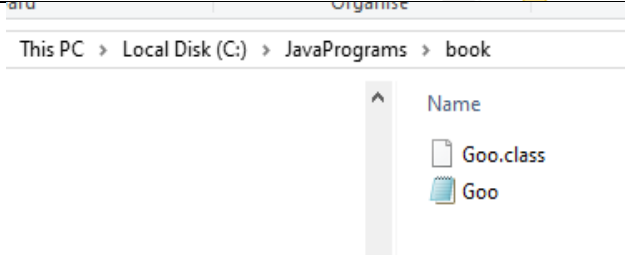
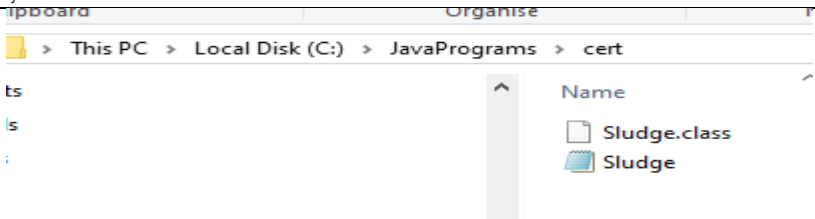
[Constructor] Difference : 10

[Method] Sum of 5 and 15 : 20

[Method] Sum of 1, 2, and 3 = 6

- Constructor chaining with `this(...)`
 - Method overloading based on parameters
 - Arithmetic operations (add/subtract)
 - Execution flow of constructors
-

Program 3 : Package (refer Lec03AnatomyJava slide 27-28)

#	File Explorer + code
1	
2	 <pre>package book; import cert.Sludge; // Import the Sludge class from the cert package public class Goo { public static void main(String[] args) { Sludge o = new Sludge(); o.testIt(); } }</pre>
3	 <pre>package cert; public class Sludge { public void testIt() { System.out.println("sludge"); } }</pre>

Program 4 : Carpet room case study

Below is a complete **Java mini project** that aligns perfectly with **Chapter 2 topics**: classes, objects, constructors, methods (including `toString()`, `equals()`), and passing parameters by object.

 Exercise Title: *Calculating Carpet Cost for Hotel Rooms*

Objective:

By completing this exercise, students will be able to:

- Create and use **classes and objects**.
 - Implement and use **constructors**.
 - Write and call **methods**, including `toString()` and `equals()`.
 - Pass **objects as parameters** to other classes.
 - Apply **object-oriented programming** concepts in a real-world scenario.
-

Instructions:

◆ Step 1: Create a `Room` class

This class will store the room's length and width, calculate its area, and compare rooms.

```
public class Room {
    private double length;
    private double width;

    public Room(double length, double width) {
        this.length = length;
        this.width = width;
    }

    public double getArea() {
        return length * width;
    }

    @Override
    public String toString() {
        return "Room [Length = " + length + ", Width = " + width + ", Area = " + getArea() + " sq. meters]";
    }

    public boolean equals(Room otherRoom) {
        return this.getArea() == otherRoom.getArea();
    }
}
```

◆ Step 2: Create a `Carpet` class

This class stores the carpet price per square meter.

```
public class Carpet {
    private double pricePerSqMeter;

    public Carpet(double price) {
        this.pricePerSqMeter = price;
    }

    public double getPrice() {
        return pricePerSqMeter;
    }

    @Override
    public String toString() {
        return "Carpet [Price per square meter = RM " + pricePerSqMeter +
    "]\n";
    }
}
```

◆ Step 3: Create a `RoomCarpet` class

This class combines a room and a carpet object, calculates total carpet cost.

```
public class RoomCarpet {
    private Room room;
    private Carpet carpet;

    public RoomCarpet(Room room, Carpet carpet) {
        this.room = room;
        this.carpet = carpet;
    }

    public double getTotalCost() {
        return room.getArea() * carpet.getPrice();
    }

    @Override
    public String toString() {
        return room.toString() + "\n" + carpet.toString() + "\nTotal Carpet
Cost = RM " + getTotalCost();
    }
}
```

◆ Step 4: Create a `Main` class to run the program

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Get room dimensions
```

```

        // Get carpet price

        // Create objects


        // Display details
        System.out.println("\n=== Room Carpet Summary ===");


        // Testing equals method


        System.out.println("\nIs this room equal to another room with the
same size?");


        // should return true

        input.close();
    }
}

```

✓ Expected Output (Sample)

```

Enter room length (in meters): 5
Enter room width (in meters): 4
Enter carpet price per square meter: RM 12.5

=== Room Carpet Summary ===
Room [Length = 5.0, Width = 4.0, Area = 20.0 sq. meters]
Carpet [Price per square meter = RM 12.5]
Total Carpet Cost = RM 250.0

Is this room equal to another room with the same size?
true

```

📖 Topics Covered from Chapter 2:

- ✓ Object & Class creation
 - ✓ Constructor & parameterized constructors
 - ✓ Method overloading
 - ✓ toString() method
 - ✓ equals() method
 - ✓ Passing objects as parameters
 - ✓ Basic encapsulation with private variables
-